

Radiola da Quebrada — prática guiada

RADIOLA DA QUEBRADA — prática guiada de flexbox

Para a dupla · programação em par · 17 passos + 1 desafio

O que vocês vão montar

A página de uma rádio de bairro: barra do topo, filtros, grade de discos, fila de reprodução, palcos e um tocador fixo no pé da tela.

O HTML já está pronto. A aparência também — cor, borda, tipo, espaço interno. O que falta é o **leiaute**, e é isso que vocês vão escrever.

As duas regras que não se quebram

1. Ninguém toca no <code>index.html</code>	Nem uma vírgula. Se a resposta parece exigir mudar o HTML, a resposta está errada
2. Só se escreve flexbox	Todo bloco deste material é propriedade de flexbox. Se você sentir vontade de usar <code>float</code> , <code>position</code> ou <code>width</code> para resolver leiaute, pare: não é por aí

Tudo que **não** é flexbox — cor, tamanho de letra, altura de caixa, reticências — **já está** no arquivo de aparência. Vocês não precisam escrever nada disso.

Onde escrever

Abram `css/estilo.css` e vão até o **fim do arquivo**. Cada passo tem um bloco para colar **no fim**, na ordem. Nunca apaguem o que já está lá.

Os papéis da dupla

Um computador só. Programação em par é duas cabeças numa tela. Fechem a segunda máquina — com as duas abertas vocês trabalham sozinhos lado a lado, e é exatamente isso que o método existe para evitar.

	PILOTO — está no teclado	NAVEGADOR — mãos longe
Antes de colar	espera	lê o passo em voz alta e diz: “isso vai no pai ou no filho?”
Na hora de colar	cola no fim do arquivo, salva, recarrega	olha a metade DEPOIS da figura
Depois de recarregar	diz o que mudou, com as próprias palavras	compara a tela com a imagem e diz se bateu
Se não bateu	desfaz o último bloco	acha a diferença entre a tela e a imagem

O papel difícil é o de navegador, porque é fácil virar passageiro. Se você está navegando e ficou calado cinco minutos, o método parou.

Rodízio — trocam de papel no fim de cada bloco

Bloco	Passos	O que fica pronto
A	1 a 4	A barra do topo
B	5 a 6	Os filtros
C	7 a 10	O cartão e a grade de discos
D	11 a 14	A fila de reprodução
E	15 a 16	Os palcos
F	17	O tocador

Três rodadas no teclado para cada um. Se um pilotou quatro blocos e o outro dois, o rodízio furou.

Como ler cada passo

Todo passo desta folha tem a mesma forma:

1. **O bloco de CSS** — é a resposta, para colar no fim do arquivo;
2. **A comparação** — uma figura só, com **ANTES** em cima e **DEPOIS** embaixo. A de baixo é o alvo: é assim que a tela de vocês tem de ficar;
3. **Por quê** — o que a propriedade faz;

4. **Confira** — a medida que muda.

As fotos são da página de verdade, tiradas do mesmo arquivo que vocês têm na mão — não são desenho. Comparem as duas metades da figura antes de colar: achar a diferença sozinho vale mais do que ler a explicação. Se a tela de vocês não ficar igual à metade de baixo, o lugar mais provável do erro é o seletor: o CSS foi para o filho quando devia ir para o pai.

Dois passos pioram a página de propósito: o 5 e o 11. Não é erro de digitação de vocês. A metade de baixo da figura vai mostrar a página feia mesmo. O conserto vem no passo seguinte, nos dois casos.

BLOCO A · A barra do topo

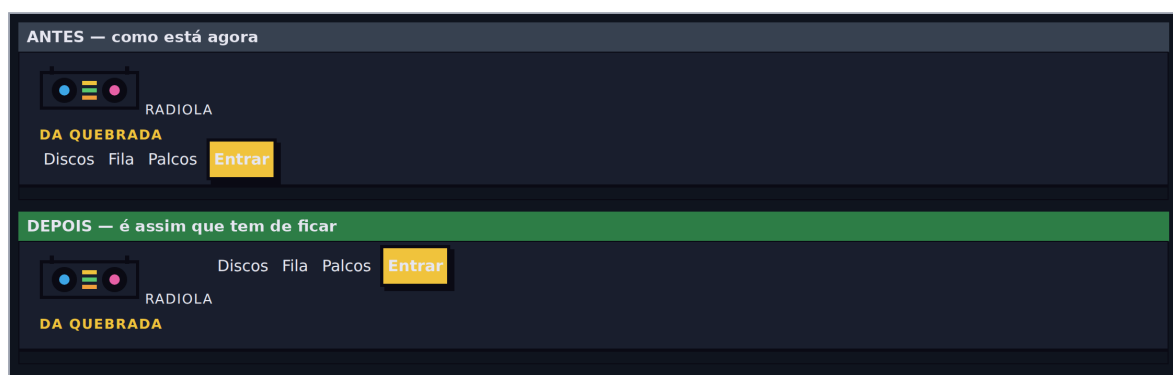
Trocamos de papel agora. Quem estava no teclado passa a navegar. Anotem: **piloto do bloco A** = _____

Passo 1 · A barra do topo vira uma linha

Propriedade: `display: flex` — vai em `.topo`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.topo{  
  display: flex;  
}
```



Por quê. Logo e menu são dois blocos: cada um ocupa a largura toda e um cai embaixo do outro. `display: flex` no PAI põe os filhos diretos lado a lado. Quem muda de comportamento são os filhos — o pai continua um bloco.

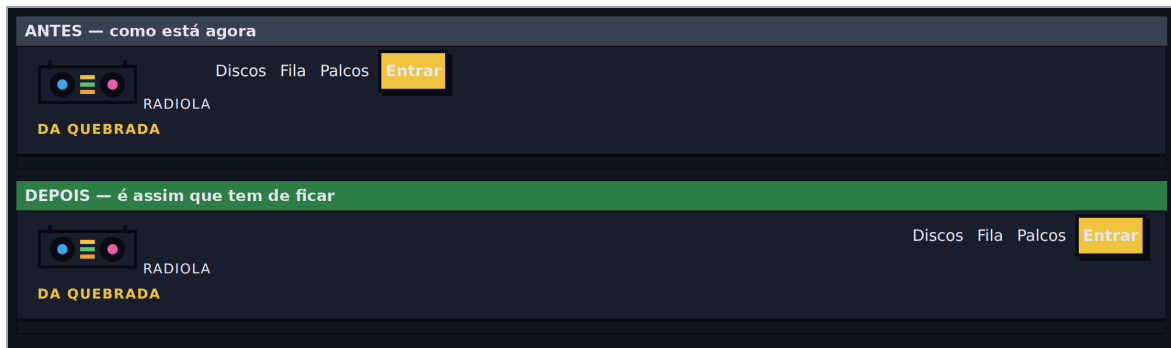
Confira. topo de `.menu` tem de **diminuir**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 2 · Logo na esquerda, menu na direita

Propriedade: justify-content: space-between — vai em .topo

O navegador lê em voz alta, o piloto cola no fim de css/estilo.css:

```
.topo{
  display: flex;
  justify-content: space-between;
}
```



Por quê. justify-content distribui no **eixo principal** — aqui, a horizontal. space-between cola o primeiro na ponta esquerda, o último na direita e divide o resto entre eles. Nada de margin-left: auto improvisado.

Confira. direita de .menu tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 3 · Tudo centralizado na altura da barra

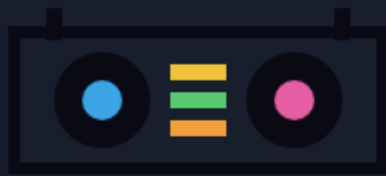
Propriedade: align-items: center — vai em .topo e .marca

O navegador lê em voz alta, o piloto cola no fim de css/estilo.css:

```
.topo{
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 10px;
}

.marca{
  display: flex;
  align-items: center;
  gap: 10px;
}
```

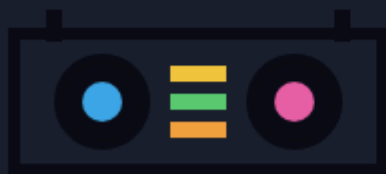
ANTES — como está agora



RADIOLA

DA QUEBRADA

DEPOIS



RADIOLA

DA QUEBRADA

Por quê. `align-items` mexe no **eixo cruzado** — com `row`, a vertical. O padrão é `stretch`: por isso o menu estava esticado do topo ao pé da barra. `center` desliga o esticamento e alinha pelo meio. A `.marca` também vira `flex`, para o desenho e o nome ficarem lado a lado.

Confira. esquerda de `.marca-nome` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 4 · Espaço entre os itens do menu — e só entre eles

Propriedade: `gap` — vai em `.menu`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.menu{  
  display: flex;  
  align-items: center;  
  gap: 18px;  
}
```

ANTES — como está agora

Discos Fila Palcos **Entrar**

DEPOIS

Discos Fila Palcos **Entrar**

Por quê. gap põe espaço **entre** os filhos. margin sobraria nas pontas e desalinharía o menu da borda; padding empurraria por dentro de cada link. Aqui a diferença é visível: compare a distância do último link até a borda direita.

Confira. largura de .menu tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

BLOCO B · Os filtros

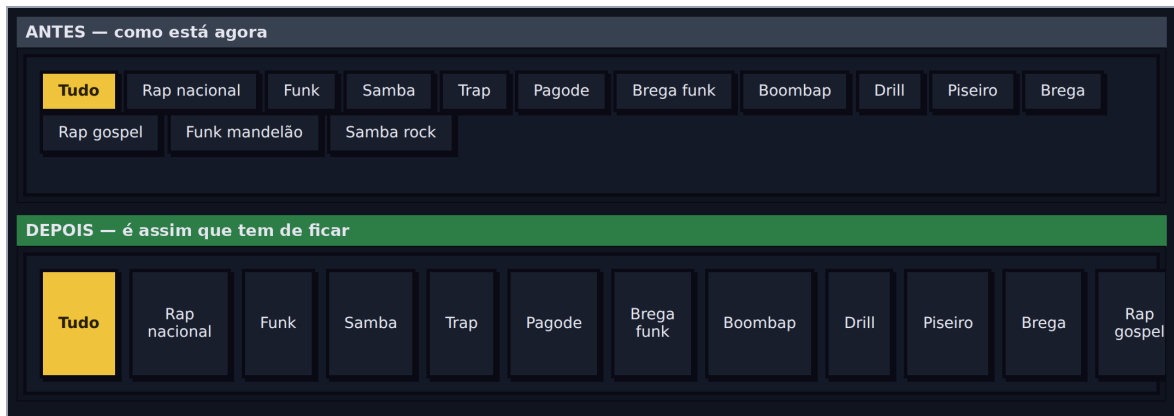
Trocam de papel agora. Quem estava no teclado passa a navegar. Anotem: piloto do bloco B = _____

Passo 5 · Os filtros entram numa linha — e são esmagados

Propriedade: display: flex (e o que ele faz de ruim) — vai em .filtros

O navegador lê em voz alta, o piloto cola no fim de css/estilo.css:

```
.filtros{  
  display: flex;  
  gap: 10px;  
}
```



Por quê. Este passo **piora** a tela de propósito, e é para você ver com os olhos. São quatorze botões. Como blocos em linha, eles já quebravam sozinhos. Ao virar flex, o container tem ordem de caber tudo em **uma linha só** e faz isso do único jeito possível: **esmagando** todo mundo. nowrap é o padrão do flex, e ninguém avisa.

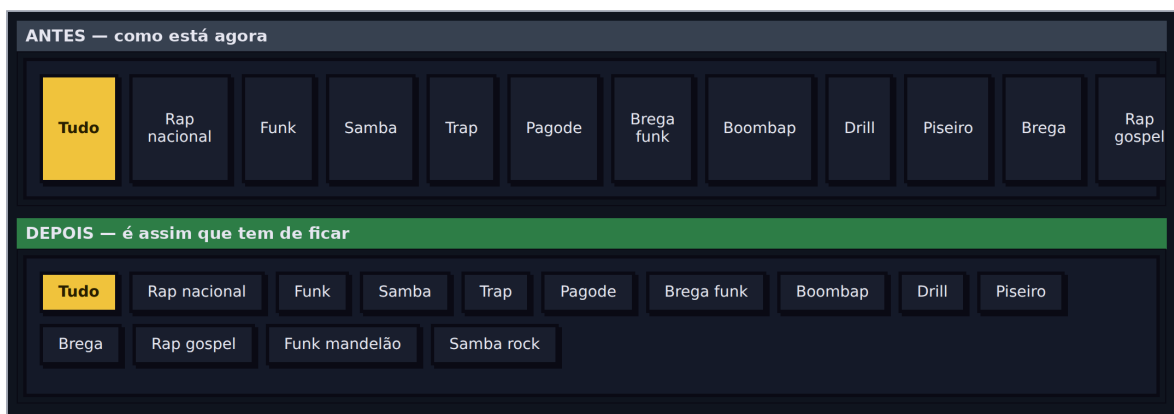
Confira. linhas de .chip tem de **diminuir**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 6 · Autorizando a quebra de linha — e distribuindo as linhas

Propriedade: flex-wrap: wrap + align-content — vai em .filtros

O navegador lê em voz alta, o piloto cola no fim de css/estilo.css:

```
.filtros{
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  align-content: flex-start;
}
```



Por quê. flex-wrap: wrap desfaz o estrago: autoriza quebrar em várias linhas em vez de espremer. E aí aparece uma propriedade que **só existe quando há mais de uma linha**: align-content, que distribui as LINHAS no eixo cruzado. Com uma

linha só ela não faz absolutamente nada — é por isso que tanta gente jura que ela “não funciona”.

Confira. linhas de `.chip` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

BLOCO C · O cartão e a grade de discos

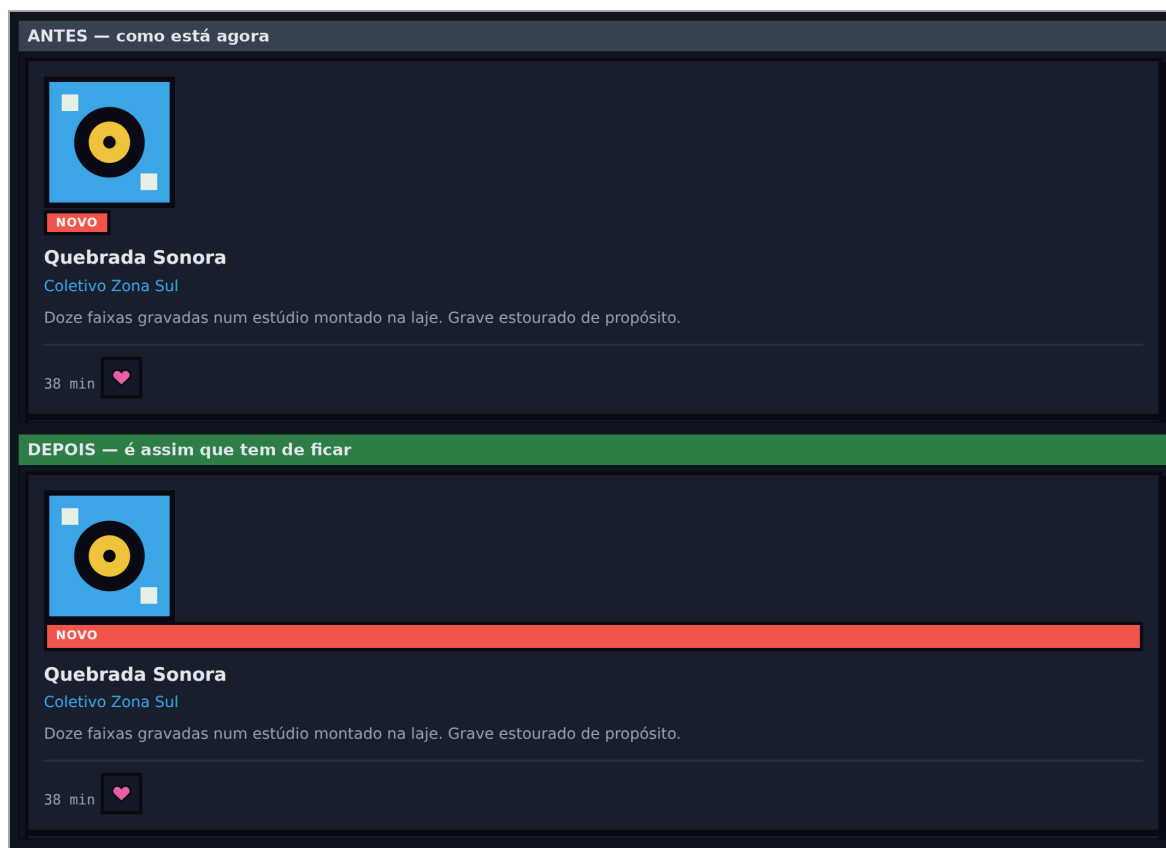
Trocamos de papel agora. Quem estava no teclado passa a navegar. Anotem: **piloto do bloco C** = _____

Passo 7 · O cartão do disco vira uma coluna

Propriedade: `flex-direction: column` — vai em `.disco`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.disco{
  display: flex;
  flex-direction: column;
}
```



Por quê. Até agora todo flex era row. column gira os eixos: o **principal** passa a ser a vertical e o **cruzado**, a horizontal. Isso inverte o significado de `justify-content` e `align-items` dentro deste cartão — e é a fonte número um de confusão em flexbox.

Repare no selo **NOVO**: ele era um span do tamanho do texto e agora atravessa o cartão inteiro. É o `align-items: stretch`, que é o padrão, agindo no eixo cruzado — que agora é o horizontal. O passo seguinte conserta isso.

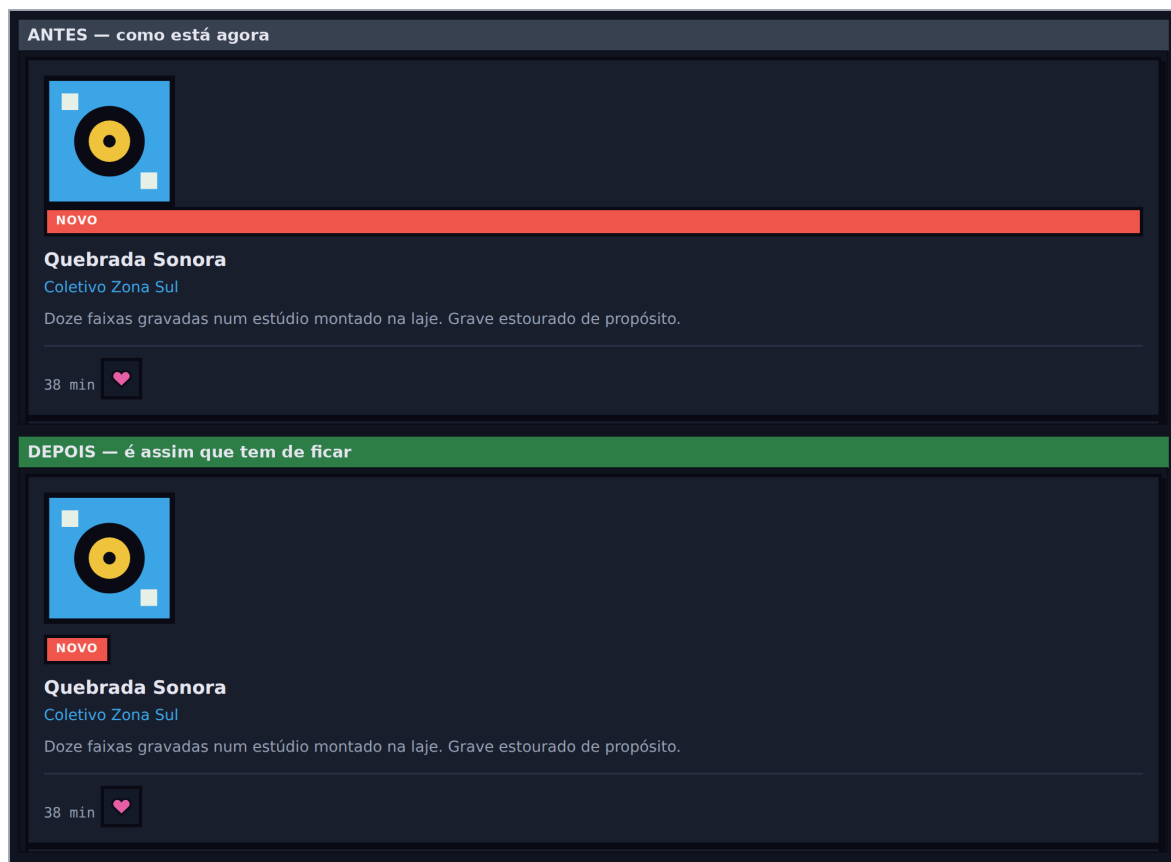
Confira. largura de `.selo` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 8 · Só o selo sai do alinhamento

Propriedade: `align-self` — vai em `.selo`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.selo{
  align-self: flex-start;
  margin-top: 10px;
}
```



Por quê. `align-items` no pai vale para todos. `align-self` é a exceção de UM item — e é propriedade do **filho**, não do container. Num cartão em coluna, o eixo cruzado é horizontal: `flex-start` faz o selo encolher para o tamanho do texto em vez de esticar de ponta a ponta.

Confira. largura de `.selo` tem de **diminuir**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

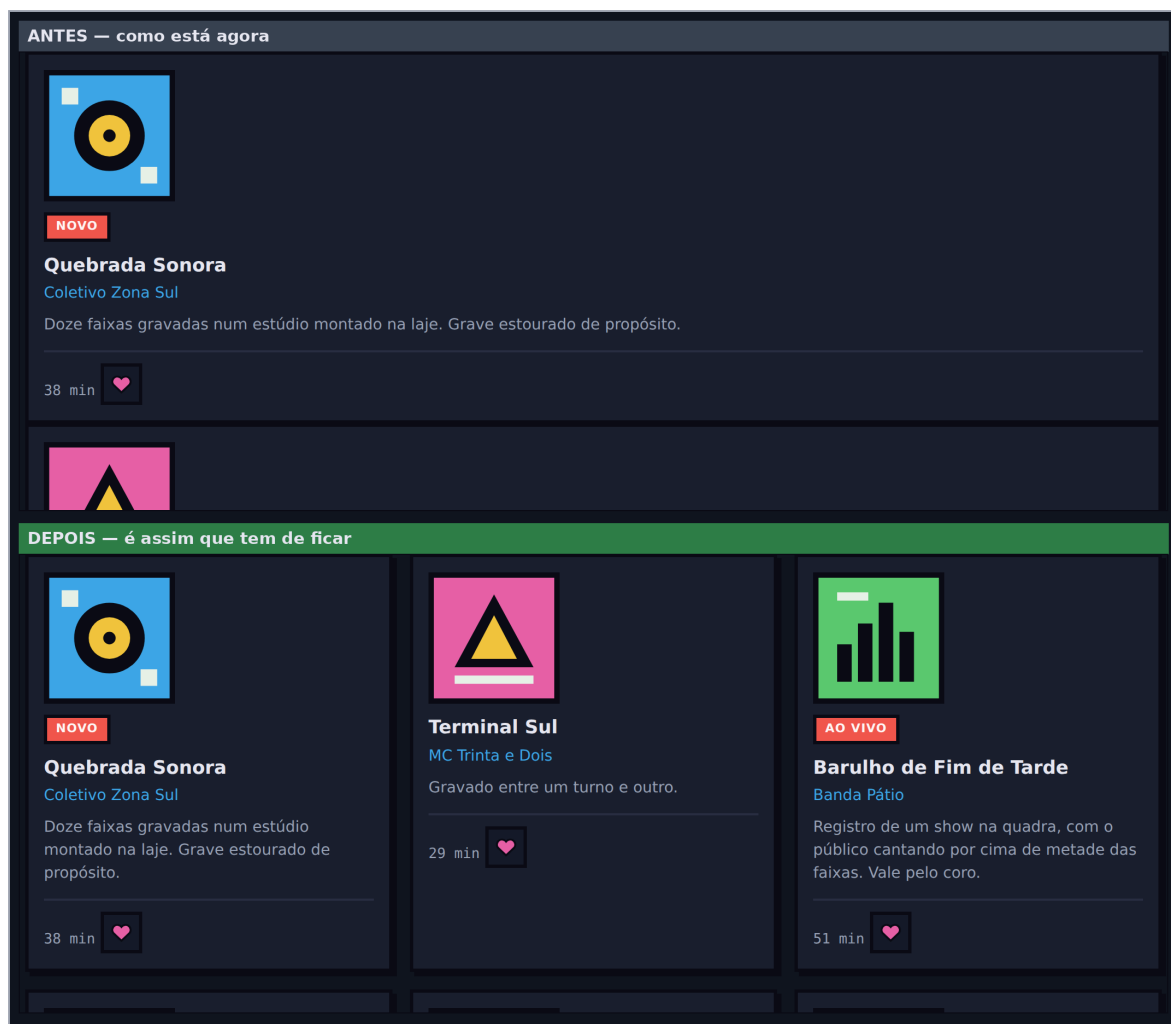
Passo 9 · Os discos entram em grade

Propriedade: flex-wrap + flex-basis — vai em .grade

O navegador lê em voz alta, o piloto cola no fim de css/estilo.css:

```
.grade{
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.disco{
  display: flex;
  flex-direction: column;
  flex: 1 1 280px;
}
```



Por quê. flex-basis é a largura de PARTIDA no eixo principal. flex: 1 1 280px lê-se: pode crescer, pode encolher, parte de 280px. Com wrap, cada linha acomoda quantos couberem a partir dessa base — grade responsiva sem nenhuma consulta de mídia.

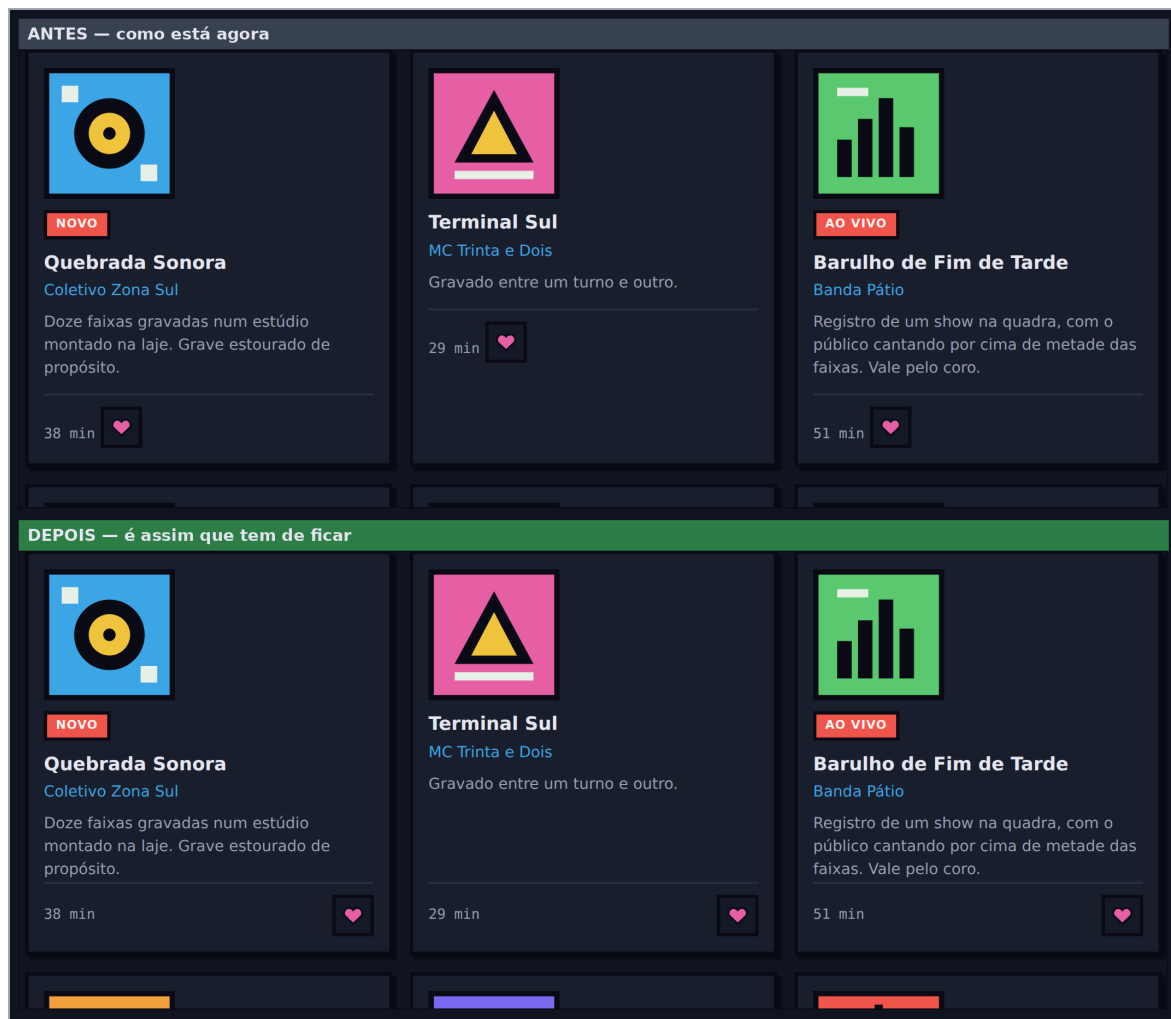
Confira. porLinha de .disco tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 10 · O rodapé do cartão desce até o fim

Propriedade: `margin-top: auto` — vai em `.disco-rod`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.disco-rod{
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: auto;
}
```



Por quê. Os textos têm tamanhos diferentes, então os rodapés ficam em alturas diferentes. Agora que os cartões dividem uma linha, eles têm **a mesma altura** (é o stretch do passo anterior agindo). Numa coluna flex, `margin-top: auto` **come todo o espaço que sobrou** e empurra o item para o fim. Sem altura fixa, sem position. Este é o truque que mais aparece em cartão de verdade.

Confira. `alinhaNaLinha` de `.disco-rod` tem de **passar a valer para todos**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

BLOCO D · A fila de reprodução

Trocam de papel agora. Quem estava no teclado passa a navegar. Anotem:
piloto do bloco D = _____

Passo 11 · A fila vira linha — e no celular ela vaza para fora da tela

Propriedade: `display: flex` na fila — vai em `.faixa`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.faixa{  
  display: flex;  
  gap: 12px;  
}
```

ANTES — como está agora

01



Barulho de Fim de Tarde Banda Pátio 4:12

02



Quebrada Sonora Coletivo Zona Sul 3:05

03



Bloco 4 (ao vivo na quadra) Meninas do Bloco
5:48

DEPOIS — é assim que tem de ficar

01



Barulho de Fim de Tarde Banda Pátio 4

02

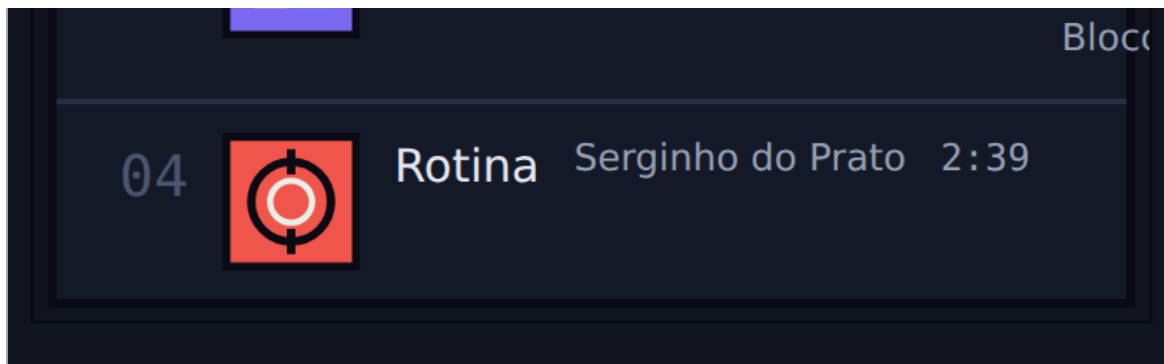


Quebrada Sonora Coletivo 3:05
Zona
Sul

03



Bloco 4 (ao vivo na quadra) Meni
do



Por quê. Número, capa, título, artista e duração entram numa linha só. Agora **aperte a janela até uns 400px**: a linha atravessa a tela e aparece barra de rolagem horizontal. Guarde essa imagem — o passo seguinte explica por que, e a explicação é a pegadinha mais famosa do flexbox.

Uma coisa que você vai ler errado por aí. É comum dizer que “no flex a imagem estica e deforma”. Eu supus isso, escrevi um passo inteiro em cima disso e **fui medir: a capa continua 48×48, antes e depois.** Uma `<img>` com `width` e `height` escritos no HTML tem tamanho definido nos dois eixos, e o flex respeita. A imagem só deforma quando alguém tira essa definição — com `width: 100%`, por exemplo. **Meça antes de acreditar, inclusive em mim.**

Confira. porLinha de `.faixa > *` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 12 · Desesticar a capa, e a pegadinha mais famosa do flexbox

Propriedade: `align-items`, `flex-shrink: 0` e `min-width: 0` — vai em `.faixa`, `.faixa-cap` e `.faixa-tit`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.faixa{
  display: flex;
  align-items: center;
  gap: 12px;
}

.faixa-cap{
  flex-shrink: 0;
}

.faixa-tit{
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

ANTES — como está agora

01



Barulho de Fim de Tarde

Banda
Pátio

4

02



Quebrada Sonora

Coletivo 3:05
Zona
Sul

03



Bloco 4 (ao vivo na quadra)

Meni
do
Bloco

04



Rotina Serginho do Prato 2:39

DEPOIS — é assim que tem de ficar

01



Barulho de Fi...

Banda
Pátio

4:12

02



Quebrada ...

Coletivo
Zona Sul

3:05

03



Bloco 4 (ao v...

Meninas
do Bloco

5:48

04



Rotina Serginho do Prato 2:39

Por quê. Por que a linha vazava: a largura mínima automática de um item flex é o tamanho do conteúdo dele. O título da faixa não quebra linha (`white-space: nowrap`, lá no arquivo de aparência), então ele se recusa a ficar menor que o texto inteiro e empurra a linha para fora da tela. `min-width: 0` desliga essa mínima automática; `overflow: hidden` com `text-overflow: ellipsis` corta o que sobrar com reticências. **Se você já brigou com um flex que não encolhia, era isto.**

`align-items: center` alinha tudo pelo meio da linha. E `flex-shrink: 0` na capa é cinto de segurança: aqui ela não encolheria nem sem ele, pelo motivo do passo anterior — mas no dia em que alguém puser `width: 100%` na imagem, essa proteção some e o `flex-shrink: 0` é o que segura.

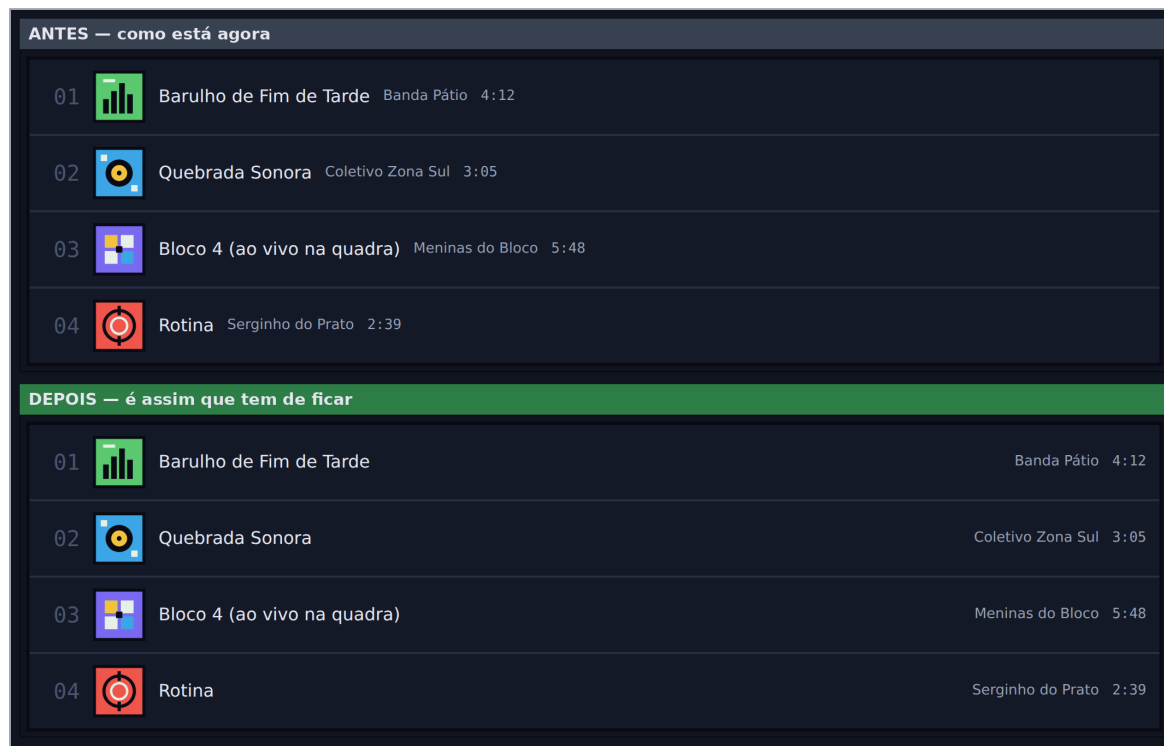
Confira. transbordoEstreito de body tem de **ir a zero**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 13 · O título da faixa ocupa o que sobrar

Propriedade: `flex-grow` (via `flex: 1`) — vai em `.faixa-tit`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.faixa-tit{  
  flex: 1;  
}
```



Por quê. `flex: 1` é atalho de `flex: 1 1 0`. O item parte de zero e **cresce** para engolir todo o espaço livre — o que empurra a duração para a ponta direita sem `margin-left: auto`. Repare que `flex-grow` reparte a SOBRA, não a largura total.

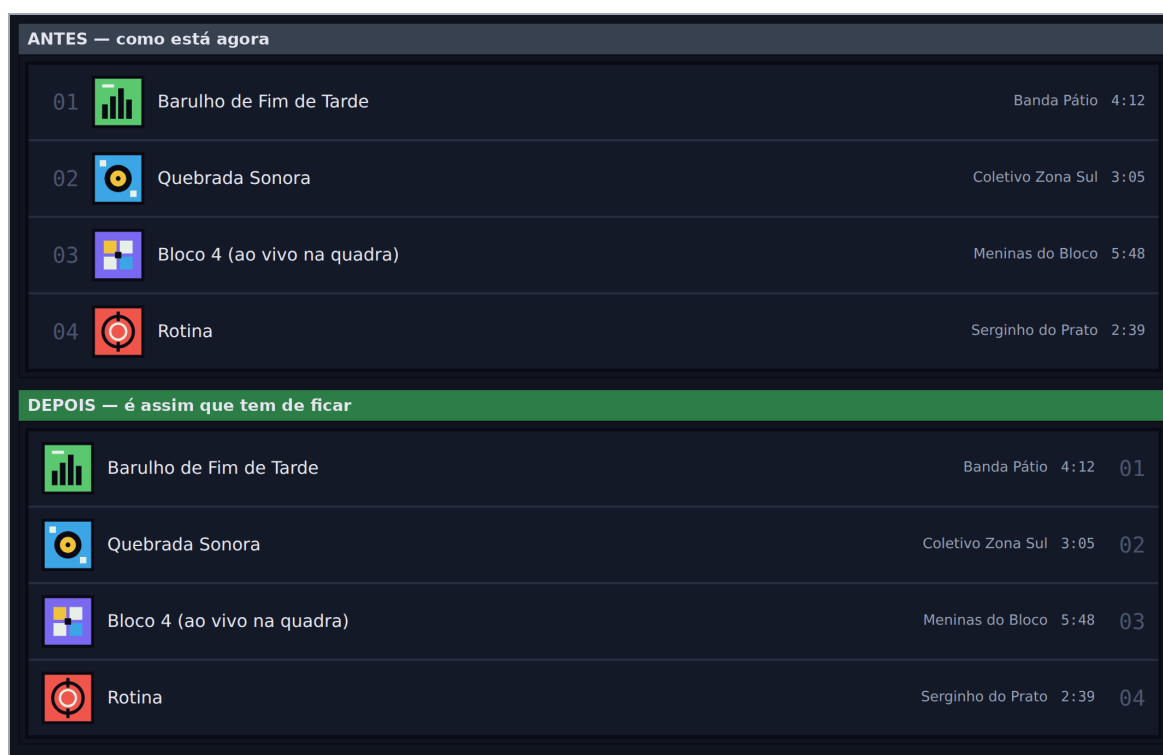
Confira. direita de `.faixa-dur` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 14 · O número da faixa vai para o fim, sem tocar no HTML

Propriedade: `order` — vai em `.faixa-n`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.faixa-n{  
  order: 1;  
}
```



Por quê. `order` muda a ordem **visual** sem mexer na marcação. Padrão é 0; quem tem número maior vai para depois. Use com cuidado: o leitor de tela e o teclado seguem o HTML, não o `order`. Se a ordem importa para entender, mude o HTML.

Confira. esquerda de `.faixa-n` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

BLOCO E · Os palcos

Trocam de papel agora. Quem estava no teclado passa a navegar. Anotem: piloto do bloco E = _____

Passo 15 · Título e artista pela linha de base

Propriedade: `align-items: baseline` — vai em `.palco`

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.palco-cab{  
  display: flex;  
  align-items: baseline;  
  gap: 10px;  
}
```



Por quê. `baseline` alinha pela linha onde as letras se apoiam, não pelo topo nem pelo meio da caixa. Com tipos de tamanhos diferentes — a hora em 26px e o título em 17px — é o único alinhamento que parece certo ao olho.

Confira. altura de `.palco-cab` tem de **diminuir**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

Passo 16 · Os palcos em linha, e as duas propriedades que quase ninguém usa

Propriedade: `flex-flow` (atalho) + `row-reverse` e `wrap-reverse` — vai em `.palcos`

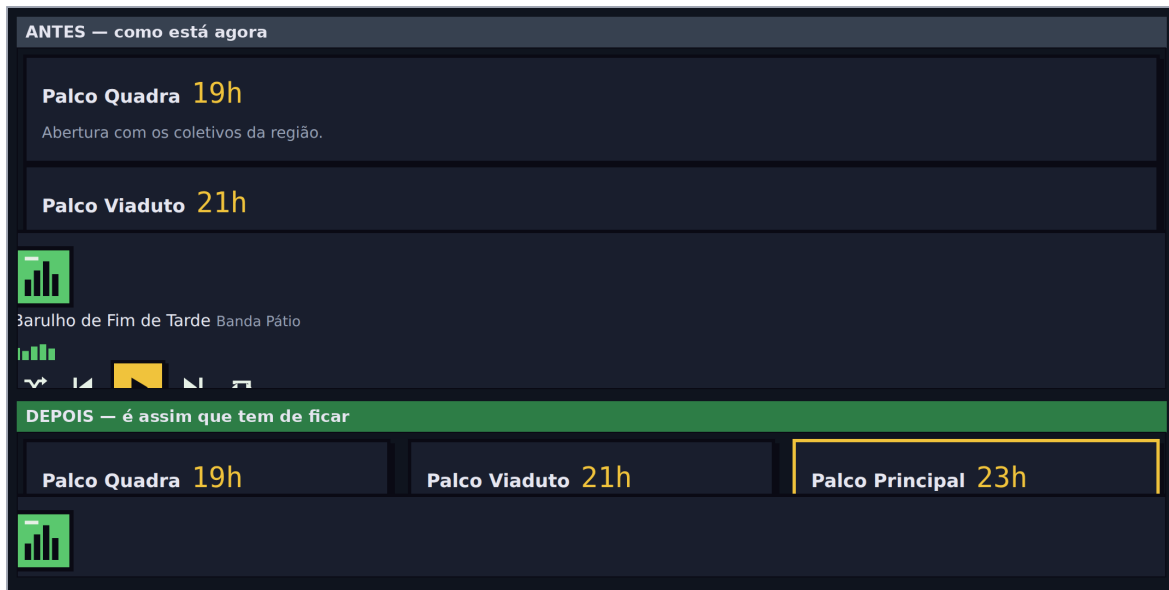
O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```

.palcos{
  display: flex;
  flex-flow: row wrap;
  gap: 16px;
}

.palco{
  display: flex;
  flex-direction: column;
  flex: 1 1 220px;
}

```



Por quê. `flex-flow: row wrap` é atalho de `flex-direction + flex-wrap`. Já `row-reverse` e `wrap-reverse` existem, funcionam, e você vai usar pouco — porque invertem a ordem visual sem inverter a do teclado, e isso confunde quem navega sem mouse. **Estão aqui para você saber que existem e por que evitá-las**, não para enfeitar.

Confira. porLinha de `.palco` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

BLOCO F · O tocador

Trocam de papel agora. Quem estava no teclado passa a navegar. Anotem: **piloto do bloco F** = _____

Passo 17 · Fechando: três blocos, um que cresce

Propriedade: o player inteiro — vai em `.player` e filhos

O navegador lê em voz alta, o piloto cola no fim de `css/estilo.css`:

```
.player{
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 12px 20px;
}

.player-agora{
  display: flex;
  align-items: center;
  gap: 12px;
  flex: 0 1 300px;
  min-width: 0;
}

.player-cap{
  flex-shrink: 0;
}

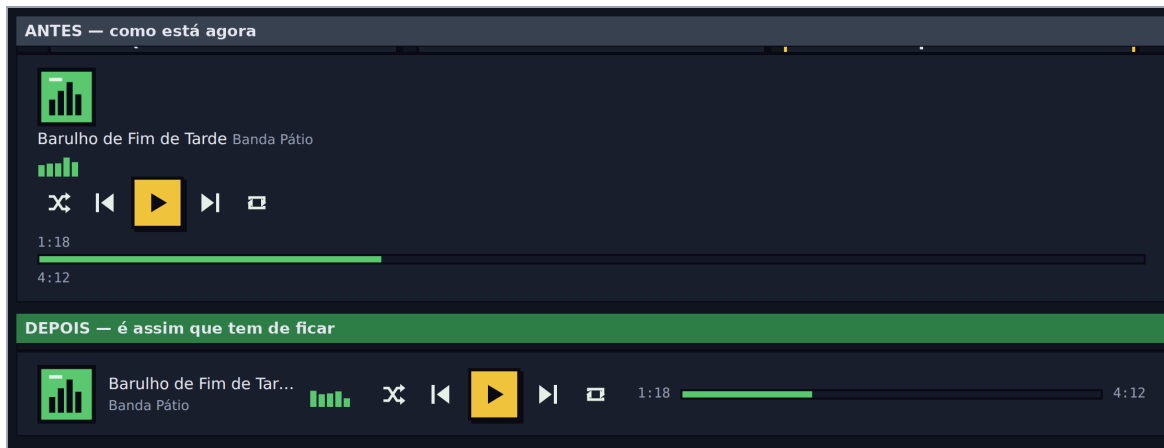
.player-txt{
  display: flex;
  flex-direction: column;
  min-width: 0;
}

.player-eq{
  flex-shrink: 0;
}

.player-controles{
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
}

.player-tempo{
  display: flex;
  align-items: center;
  gap: 10px;
  flex: 1;
}

.barra{
  flex: 1;
}
```



Por quê. O player junta tudo: a barra é uma linha com `align-items: center`; o bloco de tempo usa `flex: 1` para esticar; os controles ficam centrados com `justify-content: center`; a capa não encolhe; e o texto usa `min-width: 0` — sem isso, título longo estoura a barra, porque a largura mínima automática de um item flex é o tamanho do conteúdo.

Confira. porLinha de `.player > div` tem de **aumentar**. Se a sua tela não ficou igual à foto: o CSS foi para o pai ou para o filho?

DESAFIO FINAL — este não tem resposta aqui

Até agora vocês seguiram instruções e tinham a foto do resultado. Agora é o contrário: **está descrito o resultado, e o caminho é de vocês.**

Continuem em dupla, continuem trocando de papel, e continuem com as duas regras: **não se mexe no HTML** e **só se escreve flexbox**.

O que construir

Uma tela nova, `desafio.html + css/desafio.css`, chamada **“Meu Rolê”** — a agenda de quem vai ao festival. O HTML dessa tela **também vem pronto**, no mesmo lugar dos outros arquivos.

1 · A faixa de horários

Uma linha com **18h, 19h, 20h, 21h, 22h, 23h, 00h**. No computador eles ocupam a largura toda, com o mesmo espaço **entre** eles e **nenhuma sobra nas pontas**. No celular quebram em duas linhas, e a segunda fica **colada** na primeira, não espalhada pela altura da caixa.

2 · Três palcos com o rodapé alinhado

Três cartões lado a lado. Cada um tem título, uma lista de atrações **de tamanhos diferentes** e, no pé, um botão “Quero ir”. **Os três botões têm de ficar na mesma altura** — sem escrever nenhuma altura fixa em lugar nenhum.

3 • O cartão do seu horário

Um cartão largo com:

- a capa à esquerda, **72×72**, que não encolhe;
- o texto no meio, **ocupando toda a sobra**, cortando com reticências se for longo;
- um selo “CONFIRMADO” **alinhado no topo**, diferente de todos os outros itens da linha — **um item só**, sem mexer no alinhamento dos demais.

4 • A regra difícil

No celular, **a capa aparece embaixo do texto**. Sem tocar no HTML.

Existe uma propriedade que faz isso. Antes de usá-la, escrevam em comentário **por que ela não atrapalha quem navega por teclado**. Se não conseguirem escrever o motivo, provavelmente não deviam usá-la.

5 • Meçam antes de entregar

O que medir	Como
Os três botões na mesma altura?	inspetor do navegador → compare o y dos três
A capa continua 72×72 no celular?	inspetor → caixa do elemento
A página vaza?	aperte até 360px e veja se aparece rolagem horizontal
O texto longo corta com reticências?	troque por um título de 90 caracteres

Não existe gabarito deste desafio. Existem vários caminhos certos. O que não existe é caminho certo sem medir.

O que a dupla entrega

1. **A pasta** `_aluno` com o `css/estilo.css` completo.
2. **A folha de acompanhamento** preenchida (a próxima página).
3. **Os dois arquivos do desafio**, `desafio.html` intacto e `css/desafio.css`.

FOLHA DE ACOMPANHAMENTO

Marquem **enquanto fazem**, não no fim.

Dupla: _____ e _____

Bloco	Passos	Quem pilotou	Bateu com a foto?	Se não bateu, o que estava fora do lugar
A	1-4		() tudo () em parte	
B	5-6		() tudo () em parte	
C	7-10		() tudo () em parte	
D	11-14		() tudo () em parte	
E	15-16		() tudo () em parte	
F	17		() tudo () em parte	

Três perguntas, no fim — respondam juntos

1. Em qual passo a tela de vocês mais demorou a ficar igual à foto? Por quê?

2. Qual propriedade vocês explicariam agora para alguém que faltou?

3. Qual delas vocês ainda NÃO explicariam?

A terceira é a pergunta que vale. **“Nenhuma” não é resposta** — em dezessete passos sempre fica uma. Escrever qual é já é meio caminho.